

MicroVLA2

Code Walkthrough — Complete + Line-Level, merged

This document explains the entire **MicroVLA2** repository: what it is, how the pieces fit together, what each file does, and how every section of code works — with the exact tensor shapes that flow through the model. It is the regenerated, single-system successor to the older MicroACT+MicroVLA walkthroughs: MicroACT and all of its files have been removed, the data path is LeRobot-only, and three new learnable mechanisms are documented — a **contact-point Gaussian goal head**, **per-axis adaptive action weighting**, and **optional resistance conditioning**.

Audience: a reader who knows basic Python. No prior PyTorch / Transformers / robotics knowledge is assumed; each idea is introduced where it appears.

Project: dual-Sensapex uMp4 micromanipulation VLA (ACT action-chunking CVAE + DINOv2/Cellpose vision + frozen DistilBERT language + embodiment metadata).

Generated for: RaianSilex · bsbrl micromanipulation lab.

Table of Contents

Table of Contents	2
-------------------	---

Part I — A Complete, Line-Aware Walkthrough 4

1. Overview — What MicroVLA2 Is	4
1.1 The end-to-end pipeline	4
1.2 The big ideas	4
1.3 What each top-level directory is for	5
1.4 The two contracts	5
2. Libraries and Python Idioms	6
2.1 Third-party libraries	6
2.2 Recurring idioms	6
3. Repository Map	6
3.1 The source tree	6
3.2 Who imports whom	7
4. Tensor and Shape Reference	7
4.1 Core constants	7
4.2 The MicroVLA2 source sequence (vla_cvae.py)	8
4.3 The loss	8
5. config/vla_config.py — the single constants file	8
6. model/ — the neural network	8
6.1 backbone.py — image encoders -> tokens	9
6.2 transformer.py — attention building blocks	9
6.3 vla_cvae.py — the Vision-Language-Action CVAE	9
6.4 vla_policy.py — masked loss + metadata-aware inference	9
6.5 language_encoder.py & embodiment.py	9
6.6 finetune.py — adapting a pretrained checkpoint	10
7. data/ — turning files into tensors	10
7.1 vocab.py	10
7.2 lerobot_vla_dataset.py — the data path	10
7.3 feature_cache.py	10
8. dataset_vla/ — converters (LeRobot only)	10
8.1 convert_microact_to_lerobot.py — LeRobot v3.0	10
8.2 convert_microact_to_lerobot_v21.py — LeRobot v2.1 (OpenPI)	11

9. Training — <code>utils.py</code> and <code>train_vla.py</code>	11
9.1 <code>utils.py</code>	11
9.2 <code>train_vla.py</code>	11
10. rollout/ — deploying on the robot	11
11. Auxiliary — <code>push_to_huggingface.py</code>	12
12. The recorded data files	12
13. End-to-end tensor trace (one training step)	12
Appendix A. Glossary	12

Part II — Line-Level Code Walkthrough 13

Shape Legend	13
End-to-End Tensor Flow	13
<code>requirements.txt</code> and <code>.gitignore</code>	13
<code>utils.py</code>	13
<code>train_vla.py</code>	14
<code>config/vla_config.py</code>	14
<code>data/vocab.py</code>	14
<code>data/lerobot_vla_dataset.py</code>	15
<code>data/feature_cache.py</code>	15
<code>model/transformer.py</code>	15
<code>model/backbone.py</code>	16
<code>model/language_encoder.py</code> and <code>model/embodiment.py</code>	16
<code>model/vla_cvae.py</code>	16
<code>model/vla_policy.py</code>	17
<code>model/finetune.py</code>	17
<code>rollout/rollout.py</code> and <code>rollout/sensapex_env.py</code>	17
<code>rollout/vla_main.py</code>	17
<code>rollout/adapters/sensapex_dual.py</code>	18
<code>rollout/offline_replay.py</code>	18
<code>dataset_vla/convert_microact_to_lerobot.py</code> (+ <code>_v21</code>)	18
<code>push_to_huggingface.py</code> and package markers	18

Part I — A Complete, Line-Aware Walkthrough

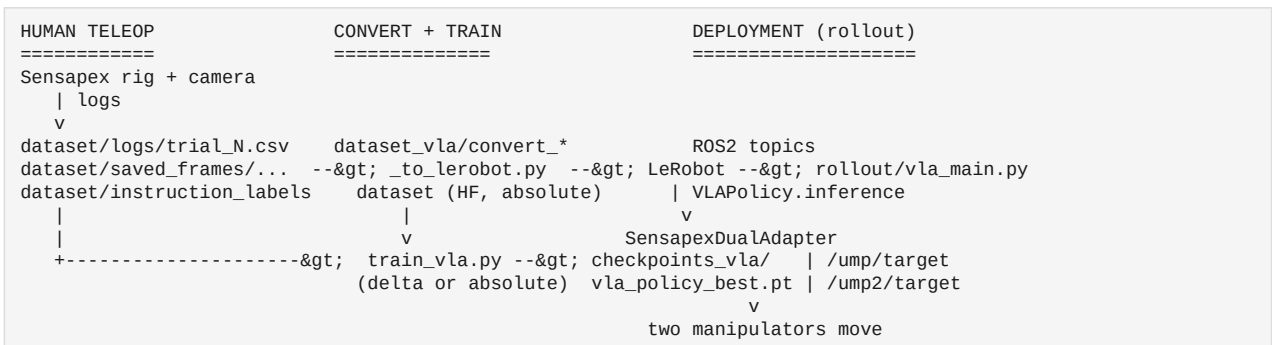
1. Overview — What MicroVLA2 Is

MicroVLA2 trains a neural-network **policy** to drive a pair of Sensapex uMp4 micromanipulators — micron-precision stages used under a microscope to manipulate cells. One microscope camera watches the workspace. The policy reads the camera image, the current stage positions, and a natural-language instruction, and predicts where the stages should move next. It learns by imitation from logged human teleoperation.

Unlike the previous repository, MicroVLA2 is a **single system**: the standalone MicroACT 8-D path has been removed. Everything is the Vision-Language-Action model — ACT action-chunking decoder, a frozen DINOv2 + Cellpose vision backbone, a frozen DistilBERT language encoder, and learned embodiment/robot/lab metadata tokens. State and action vectors are padded to a maximum of 16 dimensions with masks, so robots with different degrees of freedom can train together.

1.1 The end-to-end pipeline

Data flows through three stages — collect/convert, train, and roll out:



The robot interface never changes: the dataset always stores **absolute** Sensapex targets. Whether the model predicts absolute targets or small **deltas** is a train-time choice; at deployment, inference() always returns absolute targets.

1.2 The big ideas

(a) Action chunking

Instead of one next move, the model predicts a chunk of the next `CHUNK_SIZE = 30` moves at once and executes only the first few (`OPEN_LOOP_HORIZON = 8`) before re-predicting. This reduces compounding error and yields smooth motion. (The chunk was 100 in the old repo; 30 is far better matched to these short micromanipulation moves at ~3 Hz.)

(b) Conditional VAE for multimodal demos

Human teleop is multimodal. A small style encoder reads the ground-truth future actions during training and compresses the demonstrated style into a 32-D latent z ; the main network is conditioned on z . At inference $z = 0$ (the prior mean). A KL term keeps the latent near a standard normal so $z = 0$ is meaningful.

(c) Frozen pretrained encoders + feature cache

Vision and language are reused from frozen giant encoders (DINOv2, Cellpose/CP-SAM, DistilBERT). Only small projection layers, the metadata embeddings, the transformer trunk, and the heads are trained. Because frozen

encoders produce identical features each epoch, an on-disk feature cache (Part 7) skips both the video decode and the frozen forward passes.

(d) Heterogeneous padding + masks

Every state/action vector is padded to $\text{MAX_}_\text{DIM} = 16$ with a boolean mask of the real dims, so a 4-DOF and an 8-DOF robot share a batch and the loss only counts valid dimensions.

(e) Contact-point Gaussian goal head (new)

A dedicated decoder query predicts the episode's final reached target — the point the tip is heading toward — as a learned **mean and per-dimension variance**, trained with a Gaussian negative-log-likelihood. The variance is a calibrated confidence. The trajectory queries attend to that goal query, so the chunk is goal-conditioned in a single forward pass. This decomposition (predict where to go, then how to get there) is sample-efficient on small datasets and is backbone-agnostic.

(f) Per-axis adaptive action weighting (new)

The masked L1 over the chunk is weighted per dimension by how much that axis actually moves in the data. Near-constant axes (e.g. a fixed depth) are auto-down-weighted so they stop diluting the loss; an axis that starts moving in a future dataset is picked up automatically, with no per-rig configuration.

(g) Optional resistance conditioning (new)

If the dataset carries a per-frame pipette resistance signal (the physical signature of cell contact), the policy conditions on it via an extra source token, with modality dropout so the same checkpoint still runs when the sensor is absent. It is auto-detected end to end: the converter only writes the feature when the raw logs contain real values.

1.3 What each top-level directory is for

Directory / file	Responsibility
config/vla_config.py	All constants: shapes, hyper-parameters, dataset/feature keys, and the new feature switches. Standalone (no config.py).
model/	Backbones, transformer primitives, the VLA CVAE (with goal head), the policy wrapper, language/embodiment encoders, finetune helpers.
data/	LeRobot dataset loader (goal label, per-axis weights, optional resistance), the frozen-feature cache, the metadata vocabularies.
dataset_vla/	Converters: raw trials -> LeRobot v3.0 (MicroVLA/SmolVLA) and v2.1 (OpenPI).
rollout/	Deployment: ROS2 client, the closed-loop control loop, robot adapter, and the offline mean-collapse diagnostic.
train_vla.py / utils.py	Training entry point; seeding, optimizer, checkpoint IO, meters.
dataset/	Recorded data: trial_N.csv logs, frames, instruction_labels.csv.
checkpoints_vla/	Saved weights (.pt) — self-contained (stats + vocabs + config baked in).

1.4 The two contracts

Contract A — the per-sample dictionary. The dataset's `__getitem__` returns a dict with fixed keys: `image` (or cached `primary_feat/aux_feat`), `qpos`, `action`, `is_pad`, `state_mask`, `action_mask`, `instruction`, the five metadata ids, the new **goal** target, and **resistance** when present. The training loop and the policy both speak this dict.

Contract B — the self-contained checkpoint. A saved .pt carries weights plus the normalization stats (as buffers), the metadata vocabularies, and a config block. The config now records `chunk_size`, `goal_head`, `goal_weight`, and `use_resistance` in addition to `backbone / language backend / action space`, so rollout, resume,

and finetune all rebuild the identical architecture from the checkpoint alone.

2. Libraries and Python Idioms

2.1 Third-party libraries

Library	Why it is used
torch / torch.nn / F	Tensors, autograd, layers (Linear, Embedding, MultiheadAttention, LayerNorm, Conv2d), and the functional ops (l1_loss, interpolate).
torchvision	Pretrained ResNet18 + IntermediateLayerGetter (taps a mid-network feature map).
numpy / pandas / PIL	CSV parsing, array math, image I/O.
transformers	Frozen DistilBERT for the language channel (--language-backend hf).
lerobot	HuggingFace robot-dataset format/library. The data path. Imported lazily.
cellpose	Cell-segmentation nets reused as frozen feature extractors (CPnet U-Net, CP-SAM Transformer).
roscpy / std_msgs / sensor_msgs	ROS 2 Python client — rollout only.
reportlab	(This document generator only — not a MicroVLA2 dependency.)

2.2 Recurring idioms

- **from __future__ import annotations** — type hints stay strings; no runtime effect.
- **nn.Module subclasses** — `__init__` creates layers, `forward` defines compute; call the object, not `forward`.
- **Buffers vs parameters** — `register_buffer` stores tensors saved with the model but not trained (used for normalization stats and per-robot tables).
- **Factory functions `build_*`** — uniform construction with config-sourced defaults.
- **Lazy imports** — `lerobot/cellpose/transformers` imported inside functions so the core imports without them.
- **Context managers** — `torch.no_grad()` for inference; a `lambda` picks `bf16` autocast or `nullcontext` for `--amp`.

3. Repository Map

3.1 The source tree

```

MicroVLA2/
|-- config/vla_config.py      # ALL constants (standalone)
|-- model/
|   |-- backbone.py          # ResNet/DINOv2/Cellpose(-SAM) + proj + pos-embed + dual fusion
|   |-- transformer.py        # DETR-style encoder/decoder blocks
|   |-- language_encoder.py   # frozen DistilBERT (+ offline 'simple' hash fallback)
|   |-- embodiment.py         # 5 learned metadata tokens
|   |-- vla_cvae.py           # the model: CVAE + contact-point goal head + resistance token
|   |-- vla_policy.py         # weighted L1 + goal NLL + KL ; raw-unit inference
|   |-- finetune.py           # vocab/stats extension, partial load, freezing, LoRA
|-- data/
|   |-- vocab.py              # VocabBundle (metadata name-&id maps)
|   |-- lerobot_vla_dataset.py # LeRobot loader (+goal label, per-axis weights, resistance)
|   |-- feature_cache.py      # mmap cache of frozen-encoder features
|-- dataset_vla/
|   |-- convert_microact_to_lerobot.py # -& LeRobot v3.0 (+ optional resistance)
|   |-- convert_microact_to_lerobot_v21.py # -& LeRobot v2.1 (OpenPI)
|-- rollout/
|   |-- rollout.py            # clamp(), E-stop listener, Ctrl-C guard
|   |-- sensapex_env.py       # ROS2 client: subscribe image+state, publish targets
|   |-- vla_main.py           # closed-loop control loop (adapter-based)
|   |-- offline_replay.py     # no-hardware mean-collapse / goal diagnostic
|   |-- adapters/sensapex_dual.py # dual-Sensapex adapter + safety box
|-- train_vla.py  utils.py  push_to_huggingface.py  requirements.txt  README.md
|-- dataset/      # recorded data + instruction_labels.csv

```

3.2 Who imports whom

```

train_vla.py -& data/{lerobot_vla_dataset, vocab, feature_cache}
              -& model/vla_policy -& model/vla_cvae
              -& model/{backbone, transformer, language_encoder, embodiment}
              -& model/finetune , utils , config/vla_config
rollout/vla_main.py -& model/vla_policy , rollout/adapters/sensapex_dual
                   -& rollout/{rollout, sensapex_env}
rollout/offline_replay.py -& model/vla_policy , data/lerobot_vla_dataset

```

Dependencies point downhill from entry points to building blocks. Nothing low-level imports an entry point. The dual-Sensapex safety helpers (clamp_action_8d, limit_step) now live in the adapter itself, not in a separate rollout/main.py.

4. Tensor and Shape Reference

B is the batch size; the working width is HIDDEN_DIM = D = 512 everywhere. The transformer uses sequence-first tensors (L, B, D). A token is one 512-vector.

4.1 Core constants

Constant	Value	Role
MAX_STATE_DIM / MAX_ACTION_DIM	16	Padded width of state / each action.
CHUNK_SIZE	30	Actions predicted per inference (was 100).
HIDDEN_DIM (D)	512	Transformer / token width.
DIM_FEEDFORWARD	3200	FFN width inside each transformer layer.
ENC_LAYERS / DEC_LAYERS	4 / 7	Encoder / decoder depth.
NHEAD	8	Attention heads.
LATENT_DIM	32	CVAE style-latent z width.
KL_WEIGHT / GOAL_LOSS_WEIGHT	10.0 / 1.0	Weights on the KL and goal-NLL terms.
MAX_LANGUAGE_TOKENS	32	Text tokens fed to the language encoder.
IMAGE_HEIGHT x WIDTH	240 x 320	Model input frame size.

4.2 The MicroVLA2 source sequence (vla_cvae.py)

The non-image part of the encoder source is: a latent token, a state token, an optional resistance token, the 5 metadata tokens, and 32 language tokens — then the image tokens. The decoder runs `CHUNK_SIZE + 1` queries (the extra one is the goal query).

```
non_image = [ latent(1), qpos(1), (resistance(1)?), meta(5), language(32) ]
src        = concat(non_image, image_tokens)          -&gt; (Nfixed + 32 + S_img, B, 512)
queries    = query_embed.weight                      -&gt; (30 + 1, B, 512)  # +1 goal query
decoder(src, queries) -&gt; hs (31, B, 512)
a_hat      = action_head(hs[:30])                     -&gt; (B, 30, 16) ; * action_mask
goal_mu, lv = goal_head(hs[30]).chunk(2)              -&gt; (B, 16), (B, 16)
style_encoder (training only):
[CLS(1), qpos_tok(1), action_toks(30)] -&gt; (32, B, 512) -&gt; CLS -&gt; (mu, logvar) (B, 32)
```

4.3 The loss

```
L1 = weighted_masked_mean(|a_hat - action|)          # valid = ~is_pad AND action_mask
                                     # times per-axis weight w[dim]
KL = -0.5 * sum_latent(1 + logvar - mu^2 - exp(logvar)) (mean over batch)
GOAL = masked_mean( 0.5*((goal - goal_mu)^2 / exp(goal_lv) + goal_lv) ) # Gaussian NLL
loss = L1 + 10.0 * KL + 1.0 * GOAL
```

At inference the style encoder is skipped ($z = 0$); only `a_hat` is produced, de-normalized to raw counts, and (for the delta action space) added to the current state to give absolute targets.

5. config/vla_config.py — the single constants file

There is now one config file, and it has no logic — only named constants imported everywhere as **from config import vla_config as C**. It is fully standalone; the old `config/config.py` is gone and its still-needed constants (image size, CSV column names, ACT hyper-parameters, Cellpose-4 defaults) were folded in here.

Paths and raw-CSV layout

`REPO_ROOT`, `VLA_CKPT_DIR`, `VLA_STATS_PATH`, plus the raw-log constants used only by the converters: `CSV_STATE_COLS` (`current_*`), `CSV_ACTION_COLS` (`target_*`), `CSV_IMAGE_COL`, and the new `CSV_RESISTANCE_COL` = `'resistance_mohm'`. `DATASET_ROOT` can be overridden via the `MICROVLA_DATASET_ROOT` env var.

Shapes, model, vision, language

`MAX_STATE_DIM` = `MAX_ACTION_DIM` = 16; `CHUNK_SIZE` = 30; `HIDDEN_DIM`=512, `DIM_FEEDFORWARD`=3200, `ENC_LAYERS`=4, `DEC_LAYERS`=7, `NHEAD`=8, `LATENT_DIM`=32, `KL_WEIGHT`=10.0. `DEFAULT_BACKBONE`='dinov2_vits14+cellpose4', `BACKBONE/BACKBONE_PRETRAINED` fallbacks for the shared backbone module, the four `CELLPOSE4_*` settings, `DEFAULT_TEXT_MODEL`='distilbert-base-uncased', `LANGUAGE_BACKEND`='hf', `MAX_LANGUAGE_TOKENS`=32.

LeRobot keys, metadata, and the new feature switches

`DEFAULT_DATASET_REPO_ID`='RaianSilex/microvla_ump_dataset', `DEFAULT_ACTION_SPACE`='delta', the standard `LEROBOT_CAMERA/STATE/ACTION` keys, and the new `LEROBOT_RESISTANCE_KEY`='observation.resistance'. The five `DEFAULT_*` metadata names + `NUM_*_IDS_FALLBACK` sizes. New switches: `GOAL_HEAD`=True, `GOAL_LOSS_WEIGHT`=1.0, `GOAL_LOGVAR_MIN/MAX` (clamp for stability); `AXIS_WEIGHTING`=True, `AXIS_WEIGHT_MIN`=0.05, `AXIS_WEIGHT_MAX`=3.0; `RESISTANCE_DROPOUT`=0.3.

6. model/ — the neural network

6.1 backbone.py — image encoders -> tokens

Converts an RGB frame into $D=512$ feature tokens plus matching position embeddings. FrozenBatchNorm2d keeps BN stable at small batch sizes. Single-encoder families: ResNet18 (layer4, 1/32, 512ch), DINOv2 ViT-S/B/L (patch tokens reshaped to a grid), Cellpose-3 cyto3 U-Net encoder (256ch, 1/8), Cellpose-4/CP-SAM (256 neck + 3 readout = 259ch). Dual mode (name 'a+b') runs both, projects each with a 1x1 conv to 512, adds a learned type embedding (0=primary, 1=aux), 2x2-pools large aux grids, and concatenates tokens. PositionEmbeddingSine2D is the DETR fixed 2-D sinusoidal embedding. `encode_raw(x)` returns the raw features before the trainable projections — the hook the feature cache stores; `forward(x, primary_feat=..., aux_feat=...)` then skips the frozen encoders and runs only the trainable path.

6.2 transformer.py — attention building blocks

DETR-style, sequence-first (L,B,D), position re-added at every layer ($q=k=x+pos$, $value=x$). Encoder layer = self-attention + FFN ($512 \rightarrow 3200 \rightarrow 512$) with post-norm residuals; decoder layer = self-attention among queries + cross-attention into the encoder memory + FFN. The full Transformer encodes the source, starts the decoder from zeros, and uses `query_embed` purely as the additive query position. `build_encoder` is the encoder-only stack used as the CVAE style encoder.

6.3 vla_cvae.py — the Vision-Language-Action CVAE

Assembles the backbone + main encoder-decoder + style encoder + language encoder + embodiment conditioner. `reparameterize(mu, logvar)` lives here now (the old `cvae.py` was removed). Key additions vs the old model:

- **Goal head.** `query_embed` has `CHUNK_SIZE + 1` rows; the extra goal query's output goes through `goal_head` ($512 \rightarrow 2 \times \text{action_dim}$) to (`goal_mu`, `goal_logvar`), with `logvar` clamped. The trajectory queries see the goal query via decoder self-attention — single-pass goal-conditioning.
- **Resistance token (optional).** When `use_resistance` is set, `resistance_to_src` ($1 \rightarrow 512$) adds one source token; training applies per-sample modality dropout (zeroing its value with probability `RESISTANCE_DROPOUT`). `num_fixed_src` and `extra_src_pos` size themselves accordingly.
- **Mask-aware style + assembly.** Padded state/action dims are zeroed before projection; the source key-padding mask is False for the fixed tokens, the real lang pad for language, False for image. The action head output is multiplied by `action_mask`. Returns (`a_hat`, `goal_params`, (`mu`, `logvar`)).

6.4 vla_policy.py — masked loss + metadata-aware inference

Wraps VLACVAE with per-robot normalization tables and the loss. New buffers beyond `qpos/action mean+std`: **`action_weight_table`** (per-robot, per-dim L1 weights) and **`resistance_mean/std_table`**. `use_resistance` auto-detects from stats unless set explicitly.

- **`_compute_loss`** = per-axis-weighted masked L1 + KL + (when the goal head and goal label are present) the Gaussian goal NLL. The L1 weight per sample is gathered from `action_weight_table` by `robot_id`; goal dims are weighted the same way.
- **`inference(...)`** resolves metadata names to ids, pads `qpos` to 16, normalizes with the robot's row, runs the model ($z=0$), de-normalizes, and for the delta action space adds the current state back to return absolute targets. It accepts an optional resistance scalar, normalized with the resistance table when the model uses it.

6.5 language_encoder.py & embodiment.py

HuggingFaceTextEncoder freezes DistilBERT and trains a $768 \rightarrow 512$ projection, returning (32,B,512) tokens + a (B,32) pad mask; SimpleHashTextEncoder is a dependency-free offline fallback. EmbodimentConditioner holds five nn.Embedding tables (robot/lab/embodiment/action-type/task-family) and stacks them to (5,B,512).

6.6 finetune.py — adapting a pretrained checkpoint

`extend_vocabs` appends only unseen names at fresh ids (old ids preserved). `merge_stats` combines per-robot stats (new wins) and carries `action_weight`, resistance stats, and a `has_resistance` flag. `load_finetune_state_dict` copies exact tensors, corner-copies grown embeddings, and skips the `_table` buffers. `fill_robot_stats` writes the merged per-robot means/stds/weights/resistance into the policy buffers. `apply_freeze_mode` (none/trunk/head_only; head_only also freezes `resistance_to_src` when present) and `apply_lora` (wraps FFN `linear1/linear2` with a zero-init low-rank update) round it out.

7. data/ — turning files into tensors

7.1 vocab.py

A single frozen dataclass `VocabBundle` holding the five name->id dicts (each with :0) and an `as_dict()` for serialization. It was moved out of the (removed) episodes loader so the LeRobot loader, the policy, and finetune all share it.

7.2 lerobot_vla_dataset.py — the data path

Reads a standard LeRobot dataset and yields Contract A. The dataset stores absolute targets; this loader realizes the train-time action space: 'delta' (default) computes `action[i] = abs_target[t+i] - state_t`; 'absolute' keeps the raw targets. New responsibilities:

- **Contact-point goal label.** `__getitem__` also emits a normalized goal = the episode's final target in the chosen action representation (in delta space it shrinks toward 0 as `t` approaches the episode end).
- **Per-axis weights.** `compute_lerobot_norm_stats` keeps each dim's unclipped movement std and turns it into `action_weight` (active dims ~1, near-constant dims floored to `AXIS_WEIGHT_MIN`, padded dims 0), stored per robot.
- **Optional resistance.** `build_lerobot_vla_dataset` checks for `observation.resistance`; if present it bulk-reads it, computes resistance mean/std, sets `has_resistance=True`, and `__getitem__` emits a normalized resistance scalar.

7.3 feature_cache.py

A memmap cache of raw frozen-encoder features (fp16). Because the encoders are frozen and there is no image augmentation, each frame's features are identical every epoch — caching removes the per-step video decode and the frozen ViT passes. The cached path still runs the trainable projections, so gradients are unchanged. `load_if_valid` only reuses a cache whose `meta.json` matches (repo id, backbone, image size, frame count, `complete=True`); the trainer disables it under `--unfreeze-backbone`.

8. dataset_vla/ — converters (LeRobot only)

The non-LeRobot episode converter was removed. Two converters remain, sharing one body of instruction/region logic.

8.1 convert_microact_to_lerobot.py — LeRobot v3.0

Builds a real HuggingFace LeRobot dataset with the standard image/state/action keys and a per-trial **task** string grounded in the target cell's 3x3 frame region (`normalize_region` maps many aliases; `instruction_for` varies wording deterministically per trial). Regions come from an editable `dataset/instruction_labels.csv` (auto-scaffolded with a loud warning). Actions are stored absolute. New: it auto-detects resistance (`_dataset_has_resistance /`

_trial_resistance) and, only when the logs carry real values, adds the observation.resistance feature and writes a per-frame value (0.0 for missing cells). --no-resistance opts out.

8.2 convert_microact_to_lerobot_v21.py — LeRobot v2.1 (OpenPI)

Same content and instruction logic, but writes the older v2.1 on-disk format OpenPI/pi0 require. The on-disk version is decided by the installed lerobot (0.3.x writes v2.1; >=0.4 writes v3.0), so it is run in a separate venv with lerobot==0.3.3; the script reads back what was written and aborts if it is not v2.x.

9. Training — utils.py and train_vla.py

9.1 utils.py

set_seed; build_optimizer (two AdamW groups — backbone at lr_backbone, everything else at lr — filtering out frozen/LoRA params); save_checkpoint/load_checkpoint; AverageMeter + format_meters.

9.2 train_vla.py

LeRobot-only now (the legacy episodes branch is gone). Stages:

- **Early config read.** A --resume (or --finetune) checkpoint is inspected first so the dataset is built with the SAME action_space, chunk_size, and goal_head the run was created with — fixing the old chunk-size mismatch where rollout could not load a checkpoint trained with a different CHUNK_SIZE.
- **Dataset + split.** build_lerobot_vla_dataset(chunk_size=...), then an episode-level split (or a --holdout-lab / --holdout-robot cross-domain split).
- **Three construction modes.** resume rebuilds the exact prior architecture (incl. freeze + LoRA + use_resistance) and strict-loads; finetune extends vocabs + merges stats + partial-loads + fills stats; scratch builds fresh. use_resistance comes from the checkpoint on resume, from (pretrained OR new-data) on finetune, and from the dataset on a scratch run.
- **run_epoch** threads goal and (optional) resistance into the policy; supports bf16 --amp and the feature-cache path. **save_vla_checkpoint** records chunk_size, goal_head, goal_weight, and use_resistance in the config block so Contract B fully round-trips.

10. rollout/ — deploying on the robot

- **rollout.py** — clamp() (tolerates inverted bounds), the q+Enter E-stop listener, and a Ctrl-C-deferring context manager. (The old MicroACT RolloutArgs/parse_args were removed.)
- **sensapex_env.py** — a thin ROS2 client: subscribe to the camera + two stage live topics, publish two stage target topics, behind get_observation()/step_absolute(). Saves an atomic live-preview PNG every N frames.
- **vla_main.py** — the closed-loop control loop. load_policy rebuilds the exact architecture from the checkpoint config (now including chunk_size, goal_head, use_resistance). Each tick: observe, infer (temporal aggregation or open-loop), make safe via the adapter, optionally EMA-smooth, publish.
- **adapters/sensapex_dual.py** — the dual-Sensapex adapter. It now owns the safety box (workspace bounds + per-tick caps) and the clamp_action_8d / limit_step helpers that used to live in rollout/main.py. Swapping robots = a new adapter, no policy changes.
- **offline_replay.py** — rewritten for VLA: a no-hardware diagnostic that runs sampled frames through the policy and reports, per active axis, the normalized error ratio vs a predict-the-mean baseline (ratio ~1 = mean collapse, ratio << 1 = conditioning on inputs) plus the contact-point goal correlation. Its evaluate() takes a dataset + policy so it is unit-testable with a mock.

11. Auxiliary — push_to_huggingface.py

A small wrapper over `huggingface_hub.HfApi` to upload a folder (a trained checkpoint or a LeRobot dataset) to the Hub (`--repo-type model | dataset`). `export_onnx.py` and `resize_dataset_frames.py` from the old repo were removed.

12. The recorded data files

`dataset/logs/trial_N.csv` has one row per timestep with the 8 `current_*` (state), 8 `target_*` (action), `image_path`, and an optional `resistance_mohm` column. In the old repo resistance was always ignored; now the converter uses it automatically when populated. `dataset/instruction_labels.csv` (`trial_id`, `region`, `instruction`) is the human-curated table that grounds the language channel in each trial's target-cell region.

13. End-to-end tensor trace (one training step)

```
DataLoader batch (B=8):
  image (8,1,3,240,320) or primary_feat (8,384,17,22) [+aux] if cached
  qpos (8,16) state_mask (8,16) action (8,30,16) action_mask (8,16) is_pad (8,30)
  goal (8,16) resistance (8,1)? robot_id..task_family_id (8,) x5 instruction: list[str]
VLAPolicy.forward -&gt; VLACVAE.forward:
  (1) style (train): [CLS,qpos,actions(30)] -&gt; (32,8,512) -&gt; CLS -&gt; mu,logvar (8,32) -&gt; z
  (2) image backbone -&gt; (S_img,8,512) ; DistilBERT -&gt; (32,8,512),pad(8,32) ; meta -&gt; (5,8,512)
  (3) src = [latent,qpos,(resistance?),meta(5),lang(32),image(S_img)]
  (4) encoder(4) -&gt; memory ; decoder(7) over 31 queries -&gt; hs (31,8,512)
      a_hat = head(hs[:30]) (8,30,16)*mask ; goal_mu,lv = goal_head(hs[30]) (8,16)
  (5) loss = weighted_masked_L1 + 10*KL + 1.0*goalNLL -&gt; backward -&gt; AdamW.step()
```

Appendix A. Glossary

Term	Meaning
ACT	Action Chunking with Transformers — predict a chunk of future actions with a CVAE transformer.
Action chunk	The <code>CHUNK_SIZE = 30</code> future actions predicted in one forward pass.
Contact-point goal head	A decoder query predicting the episode's final target as a learned Gaussian (mean + per-dim variance), trained with NLL; goal-conditions the chunk.
Per-axis weighting	Data-driven per-dimension L1 weights; near-constant axes down-weighted, active axes ~1.
Resistance conditioning	Optional per-frame contact signal fed as a source token, with modality dropout for absence.
CVAE / latent z	Conditional VAE; the 32-D style code sampled differentially as $\mu + \epsilon \cdot \text{std}$.
KL divergence	Regularizer pulling the latent toward $N(0,I)$ so $z=0$ is a sensible inference default.
Token	One 512-vector the transformer processes.
Backbone / dual encoder	Frozen pretrained image encoder(s); dual = generalist (DINOv2) + specialist (Cellpose) concatenated.
Delta vs absolute	Whether actions are predicted relative to the current state or as absolute targets; always absolute at the robot.
Mask (state/action)	Boolean flags marking real padded dimensions so heterogeneous robots share a batch.
Buffer	A tensor saved with the model but not trained — normalization stats and per-robot tables.
Feature cache	On-disk memmap of frozen-encoder outputs to skip recomputation each epoch.
LoRA	Low-Rank Adaptation — train a tiny low-rank add-on instead of the full weights.
LeRobot	HuggingFace's standard robot-dataset format/library — the only data path.

Part II — Line-Level Code Walkthrough

Section-by-section walkthrough of every current source file, with real code excerpts (file:line labels) and explicit tensor shapes. MicroVLA2 is one system, so there is one config, one CVAE, one policy, and one training/rollout path.

Shape Legend

- B = batch. N = cameras (1). H, W = 240, 320. D = HIDDEN_DIM = 512. L_dim = LATENT_DIM = 32.
- k = CHUNK_SIZE = 30 future timesteps. State/action are padded to MAX_*_DIM = 16 with masks.
- S_img = image tokens. ResNet18 ~80; DINOv2 ViT-S/14 374; Cellpose-3 300 (pooled); CP-SAM a small grid. Dual mode concatenates two encoders' tokens.
- L_lang = 32 language tokens. L_meta = 5 metadata tokens. Transformer is sequence-first (L,B,D).

End-to-End Tensor Flow

- Sample: image (1,3,240,320) [or primary_feat/aux_feat if cached], qpos (16,), state_mask (16,), action (30,16), action_mask (16,), is_pad (30,), goal (16,), resistance (1,)?, 5 long id tensors, and an instruction string.
- Style (train only): [CLS, qpos_tok, action_toks] (B,32,512) -> CLS -> mu,logvar (B,32) -> z.
- Source: [latent(1), qpos(1), (resistance(1)?), meta(5), language(32), image(S_img)].
- Decoder: 31 queries -> (31,B,512); action_head on first 30 -> (B,30,16)*mask; goal_head on the 31st -> (B,16) mean + (B,16) logvar.
- Loss: per-axis-weighted masked L1 + 10*KL + 1.0*goal-NLL. Inference: z=0, de-normalize, delta+state -> absolute targets.

requirements.txt and .gitignore

requirements.txt declares torch/torchvision, numpy/pandas/pillow, transformers (DistilBERT), lerobot (the data path), and cellpose (only for cellpose backbones). lerobot/cellpose are imported lazily so the core imports without them. .gitignore excludes checkpoints, checkpoints_vla/, *.pt/*.pkl, the dataset frames/logs, and the microvla_live.png rollout preview.

utils.py

Shared training helpers: seeding, the two-group optimizer, checkpoint IO, and a running-mean meter.

utils.py:21-36 - build_optimizer

```
backbone_params = list(policy.model.backbone.parameters())
backbone_ids = {id(p) for p in backbone_params}
other_params = [p for p in policy.parameters() if id(p) not in backbone_ids]
param_groups = [
    {"params": [p for p in backbone_params if p.requires_grad], "lr": lr_backbone},
    {"params": [p for p in other_params if p.requires_grad], "lr": lr},
]
return torch.optim.AdamW(param_groups, lr=lr, weight_decay=weight_decay)
```

- policy.model.backbone works because VLAPolicy.model = VLACVAE nests the image backbone there.
- Frozen params (requires_grad=False) are filtered out, so AdamW allocates no moments for them.

- `save_checkpoint/load_checkpoint` store the policy state_dict (weights + buffers) and optional optimizer/epoch/best_val; AverageMeter weights each batch by its size.

train_vla.py

The single training entry point. Each batch carries the full VLA inputs plus goal and optional resistance; the checkpoint embeds stats + vocabs + a config block.

train_vla.py:251-266 - read architecture from the checkpoint BEFORE building the dataset

```
resume_ckpt = None
if args.resume is not None and args.resume.exists():
    resume_ckpt = torch.load(args.resume, map_location=device, weights_only=False)
    rcfg0 = resume_ckpt.get("config", {})
    for attr in ("action_space", "chunk_size", "backbone", "goal_head"):
        if attr in rcfg0: setattr(args, attr, rcfg0[attr])
elif args.finetune is not None and args.finetune.exists():
    fcfg = torch.load(args.finetune, map_location="cpu", weights_only=False).get("config", {})
    for attr in ("action_space", "chunk_size", "goal_head"):
        if attr in fcfg: setattr(args, attr, fcfg[attr])
```

- Reading `chunk_size` from the checkpoint first means the dataset is built with the SAME chunk length the weights expect — fixing the old mismatch where a 30-chunk checkpoint could not be loaded by a 100-chunk default.

train_vla.py:131-199 - run_epoch threads goal + optional resistance

```
goal = batch["goal"].to(device, non_blocking=True)
resistance = batch["resistance"].to(device) if "resistance" in batch else None
...
loss_dict = policy(image, qpos, instructions, robot_id, lab_id, embodiment_id,
                    action_type_id, task_family_id, state_mask=state_mask,
                    action_mask=action_mask, actions=action, is_pad=is_pad,
                    goal=goal, resistance=resistance,
                    img_primary_feat=img_primary_feat, img_aux_feat=img_aux_feat)
```

train_vla.py:197-233 - save_vla_checkpoint config block (Contract B)

```
"config": {
    "max_state_dim": C.MAX_STATE_DIM, "max_action_dim": C.MAX_ACTION_DIM,
    "chunk_size": int(args.chunk_size), "backbone": args.backbone,
    "language_backend": args.language_backend, "text_model": args.text_model,
    "action_space": args.action_space, "goal_head": bool(args.goal_head),
    "goal_weight": float(args.goal_weight), "use_resistance": bool(policy.use_resistance),
    "freeze_mode": args.freeze_mode, "lora_r": int(args.lora_r), ... }
```

config/vla_config.py

One standalone constants module. New blocks beyond the inherited ACT constants:

config/vla_config.py - new feature switches

```
CHUNK_SIZE = 30
GOAL_HEAD = True ; GOAL_LOSS_WEIGHT = 1.0 ; GOAL_LOGVAR_MIN = -6.0 ; GOAL_LOGVAR_MAX = 4.0
AXIS_WEIGHTING = True ; AXIS_WEIGHT_MIN = 0.05 ; AXIS_WEIGHT_MAX = 3.0
RESISTANCE_DROPOUT = 0.3
LEROBOT_RESISTANCE_KEY = "observation.resistance"
CSV_RESISTANCE_COL = "resistance_mohm"
```

data/vocab.py

data/vocab.py:14-28 - VocabBundle

```
@dataclass(frozen=True)
class VocabBundle:
    robot_ids: Dict[str, int] ; lab_ids: Dict[str, int]
    embodiment_ids: Dict[str, int] ; action_type_ids: Dict[str, int]
    task_family_ids: Dict[str, int]
    def as_dict(self) -> dict: ...
```

Five name->int maps (each with :0). Frozen so it is hashable and safely shared between the dataset, policy, and checkpoint.

data/lerobot_vla_dataset.py

Reads a LeRobot dataset and yields Contract A — now with the goal label, per-axis weights, and optional resistance.

data/lerobot_vla_dataset.py:66-83 - per-axis weights from movement std

```
def _axis_weights(raw_std, action_dim):
    weight = np.zeros(C.MAX_ACTION_DIM, dtype=np.float32)
    if not C.AXIS_WEIGHTING: weight[:action_dim] = 1.0 ; return weight
    rs = raw_std[:action_dim].astype(np.float64)
    active = rs > 1e-6 * (rs.max() + 1e-9)
    ref = np.median(rs[active]) if active.any() else 1.0
    weight[:action_dim] = np.clip(rs / ref, C.AXIS_WEIGHT_MIN, C.AXIS_WEIGHT_MAX)
    return weight
```

- Active axes land near 1.0; a near-constant axis (e.g. a fixed depth) is floored to AXIS_WEIGHT_MIN so it stops diluting the loss; padded dims get 0.

data/lerobot_vla_dataset.py:237-256 - goal label (+ optional resistance) in __getitem__

```
abs_final = self.actions_all[g0 + ep.length - 1]      # the contact point
if self.action_space == "delta":
    chunk = abs_targets - base ; goal_raw = abs_final - base[0]
else:
    chunk = abs_targets ; goal_raw = abs_final
goal[: self.action_dim] = goal_raw
goal_n = (goal - robot_stats["action_mean"]) / robot_stats["action_std"]
...
if self.has_resistance:
    r = (float(self.resistance_all[g]) - rs["resistance_mean"]) / rs["resistance_std"]
    sample["resistance"] = torch.tensor([r], dtype=torch.float32)
```

- The goal is the episode's final target in the same action representation, normalized by the action stats. In delta space it shrinks toward 0 as t nears the end.
- build_lerobot_vla_dataset detects observation.resistance in the dataset columns and, when present, bulk-reads it and sets has_resistance.

data/feature_cache.py

Memmap cache of raw frozen-encoder features (fp16). get(g) returns (primary_feat, aux_feat|None) as fp32. build runs backbone.encode_raw over every frame once and only flips meta.json complete=True at the very end; load_if_valid reuses a cache only if its metadata matches the run. Invalid under --unfreeze-backbone (enforced by the trainer).

model/transformer.py

model/transformer.py:62-73 - encoder layer (DETR: position on Q/K only)

```
q = k = _with_pos(src, pos)                                # value = src
src2 = self.self_attn(q, k, src, key_padding_mask=src_key_padding_mask)[0]
src = self.norm1(src + self.dropout1(src2))
src2 = self.linear2(self.dropout(self.activation(self.linear1(src)))) # 512->3200->512
src = self.norm2(src + self.dropout2(src2))
```


The decoder layer adds self-attention among queries, cross-attention into the encoder memory (respecting the source padding mask), then the FFN. `Transformer.forward` starts the decoder from zeros and uses `query_embed` purely as the additive query position.

model/backbone.py

Image encoders + 1x1 projection to 512 + 2-D sinusoidal positions, single or dual.

model/backbone.py:594-606 - encode_raw (the feature-cache hook)

```
@torch.no_grad()
def encode_raw(self, x):
    feat_p = self._primary_feat(x)
    feat_a = self._encoder_feat(self.aux_name, x) if self.is_dual else None
    return feat_p, feat_a
```

model/backbone.py:608-650 - forward (single -> 4D, dual -> tokens; cached path skips encoders)

```
feat_p = primary_feat if primary_feat is not None else self._primary_feat(x)
feat_p = self.input_proj(feat_p) ; pos_p = self.pos_embed(feat_p)
if not self.is_dual: return feat_p, pos_p
feat_a = aux_feat if aux_feat is not None else self._encoder_feat(self.aux_name, x)
if min(feat_a.shape[-2:]) >= 16: feat_a = self.aux_pool(feat_a)
feat_p = feat_p + self.type_embed.weight[0].view(1,512,1,1) # learned per-encoder tag
feat_a = feat_a + self.type_embed.weight[1].view(1,512,1,1)
tokens = torch.cat([feat_p.flatten(2).permute(2,0,1), feat_a.flatten(2).permute(2,0,1)], dim=0)
```

model/language_encoder.py and model/embodiment.py

model/language_encoder.py:60-75 - frozen DistilBERT -> (32,B,512) + pad mask

```
with torch.no_grad():
    out = self.text_model(**enc).last_hidden_state # (B,32,768) frozen
    tokens = self.proj(out).permute(1, 0, 2).contiguous() # (32,B,512)
    pad_mask = ~enc["attention_mask"].bool() # (B,32) True where PAD
```

EmbodimentConditioner holds five nn.Embedding tables and torch.stacks the looked-up ids into (5,B,512).

model/vla_cvae.py

The model. reparameterize lives here now. The two structural additions are the goal head and the optional resistance token.

model/vla_cvae.py:104-119 - decoder queries (+1 goal query) and the heads

```
num_queries = chunk_size + (1 if self.goal_head_enabled else 0)
self.query_embed = nn.Embedding(num_queries, hidden_dim)
self.action_head = nn.Linear(hidden_dim, action_dim) # 512 -> 16
if self.goal_head_enabled:
    self.goal_head = nn.Linear(hidden_dim, 2 * action_dim) # mu | logvar
```

model/vla_cvae.py:196-219 - resistance token + dropout, then decode + split heads

```
if self.use_resistance:
    if resistance is None: resistance = torch.zeros(B, 1, device=device, dtype=qpos.dtype)
    elif self.training and self.resistance_dropout > 0:
        keep = (torch.rand(B,1,device=device) >= self.resistance_dropout).float()
        resistance = resistance * keep
    fixed_toks.append(self.resistance_to_src(resistance).unsqueeze(0))
...
hs = self.transformer(src, src_pos, self.query_embed.weight, src_key_padding_mask=mask)
a_hat = self.action_head(hs[:self.chunk_size].transpose(0,1)) * action_mask
goal_mu, goal_logvar = self.goal_head(hs[self.chunk_size]).chunk(2, dim=-1)
goal_logvar = goal_logvar.clamp(C.GOAL_LOGVAR_MIN, C.GOAL_LOGVAR_MAX)
```

- The trajectory queries attend to the goal query through the decoder self-attention, so the chunk is goal-conditioned in one pass. forward returns (a_hat, goal_params, (mu, logvar)).

model/vla_policy.py

model/vla_policy.py:56-67 - new per-robot buffers (weights + resistance)

```
self.register_buffer("action_weight_table", torch.ones(n_robots, C.MAX_ACTION_DIM))
self.register_buffer("resistance_mean_table", torch.zeros(n_robots, 1))
self.register_buffer("resistance_std_table", torch.ones(n_robots, 1))
... # filled per robot from stats["by_robot"], incl. action_weight / resistance_mean/std
```

model/vla_policy.py:147-182 - per-axis-weighted L1 + KL + Gaussian goal NLL

```
weight = valid * axis_w.unsqueeze(1) # (B,k,A)
l1 = (l1_unreduced * weight).sum() / weight.sum().clamp_min(1.0)
kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp()).sum(dim=-1).mean()
total = l1 + self.kl_weight * kl
if goal_params is not None and goal is not None:
    goal_mu, goal_logvar = goal_params ; var = goal_logvar.exp()
    nll = 0.5 * ((goal - goal_mu).pow(2) / var + goal_logvar) # variance is learned
    gmask = action_mask.float() * axis_w
    total = total + self.goal_weight * (nll * gmask).sum() / gmask.sum().clamp_min(1.0)
```

- `axis_w = action_weight_table[robot_id]` selects the per-robot, per-dim weights; goal dims are weighted the same way. `inference()` adds an optional resistance scalar (normalized via the resistance table) and, for delta, adds the current state back to return absolute targets.

model/finetune.py

Adapting a pretrained checkpoint to new data without scrambling learned embeddings.

model/finetune.py:132-158 - load_finetune_state_dict (grown-embedding corner copy)

```
if target.shape == v.shape: new_state[k] = v ; matched.append(k)
elif target.dim() == v.dim() and all(t &gt;= s for t,s in zip(target.shape, v.shape)):
    grown = target.clone()
    grown[tuple(slice(0, s) for s in v.shape)] = v # old into the leading corner
    new_state[k] = grown ; partial.append(...) # new rows keep fresh init
```

- `merge_stats` carries `action_weight`, `resistance` stats, and `has_resistance`; `fill_robot_stats` writes them into the policy buffers. `apply_freeze_mode head_only` also freezes `resistance_to_src` when present. `apply_lora` wraps FFN `linear1/linear2` with a zero-init low-rank update.

rollout/rollout.py and rollout/sensapex_env.py

`rollout.py` is the slimmed shared layer: `clamp()` (inverted-bounds tolerant), the `q+Enter` E-stop listener, and a `Ctrl-C`-deferring context manager. `sensapex_env.py` is the ROS2 client: subscribe to `/camera/image/compressed`, `/ump/live`, `/ump2/live`; publish two `Int32MultiArray [x,y,z,d,speed]` targets; `get_observation()` returns the 8-D state + RGB image; a live preview PNG is written atomically (temp file then `os.replace`).

rollout/vla_main.py

rollout/vla_main.py:106-141 - rebuild the exact architecture from the checkpoint config

```
policy = build_vla_policy(
    stats=ckpt["stats"], vocabs=ckpt["vocabs"], backbone_name=backbone,
    freeze_backbone=freeze_backbone, language_backend=language_backend,
    text_model_name=text_model, action_space=action_space,
    chunk_size=int(ckpt_config.get("chunk_size", C.CHUNK_SIZE)),
    goal_head=bool(ckpt_config.get("goal_head", C.GOAL_HEAD)),
    use_resistance=bool(ckpt_config.get("use_resistance", False)),
).to(device)
policy.load_state_dict(ckpt["policy"])
```

The loop is unchanged in spirit: observe via the adapter, infer (temporal aggregation or open-loop), `adapter.safe_command`, optional EMA, `adapter.publish`; `q+Enter` holds position.

rollout/adapters/sensapex_dual.py

Owns the dual-Sensapex specifics the rig-agnostic policy must not know. The workspace bounds, per-tick caps, and the clamp_action_8d / limit_step helpers now live here (they used to be in the removed rollout/main.py).

rollout/adapters/sensapex_dual.py:67-105 - the adapter contract

```
class SensapexDualAdapter:
    robot_id = C.DEFAULT_ROBOT_ID ; ... ; state_dim = 8 ; action_dim = 8
    def safe_command(self, state_8d, action_8d):
        return limit_step(state_8d, clamp_action_8d(action_8d))
    def publish(self, command_8d): self.env.step_absolute(command_8d)
    def hold_current(self): self.publish(self.get_observation().state.astype(np.float32).copy())
```

rollout/offline_replay.py

A no-hardware diagnostic, rewritten for VLA and decoupled so it is unit-testable.

rollout/offline_replay.py:52-90 - normalized error ratio vs the predict-the-mean baseline

```
a_hat, goal_params, _ = policy.model(image, batch["qpos"], batch["instruction"], ...)
gt = batch["action"] # normalized
valid = (~batch["is_pad"]).unsqueeze(-1).float() * batch["action_mask"].unsqueeze(1).float()
err_policy = ((a_hat - gt).abs() * valid).sum((0,1)) / denom
err_mean = (gt.abs() * valid).sum((0,1)) / denom # baseline = predict 0 (the mean)
ratio = err_policy / err_mean.clamp_min(1e-6)
```

- ratio ~1.0 => mean-collapse (inputs ignored); ratio << 1.0 => the policy is conditioning. It also reports corr(true goal, predicted goal_mu) per active axis. evaluate() takes a dataset + policy, so it runs against the real LeRobot dataset or a mock.

dataset_vla/convert_microact_to_lerobot.py (+ _v21)

Builds a real LeRobot dataset with grounded, varied per-trial language and (new) auto-detected resistance.

dataset_vla/convert_microact_to_lerobot.py:232-253 - resistance auto-detection

```
def _trial_resistance(df):
    if RESISTANCE_COL not in df.columns: return None
    col = pd.to_numeric(df[RESISTANCE_COL], errors="coerce")
    if not np.isfinite(col.to_numpy(np.float64)).any(): return None
    return col.fillna(0.0).to_numpy(np.float32)

def _dataset_has_resistance(csv_files): ... # True if ANY trial carries real values
```

- When resistance is found, the feature C.LEROBOT_RESISTANCE_KEY is added and a per-frame value is written (0.0 for missing cells). normalize_region/instruction_for build grounded, varied task strings from a 3x3 region label. The v2.1 converter shares this logic and only differs in on-disk format (OpenPI).

push_to_huggingface.py and package markers

push_to_huggingface.py wraps huggingface_hub.HfApi to upload a checkpoint or LeRobot dataset folder (--repo-type model | dataset). The empty config/__init__.py, data/__init__.py, model/__init__.py mark importable packages; rollout/__init__.py and rollout/adapters/__init__.py carry one-line docstrings.

Coverage: this walkthrough reproduced and explained code from every current source file in the MicroVLA2 repository — the config module; all four data modules; all eight model modules; train_vla.py and utils.py; all five rollout modules plus the adapter; the two dataset_vla converters; push_to_huggingface.py; and the package markers.